

Object-based architectures:

Here we envisage each component in distributed systems as an object. In object-based architecture (Figure 3), we see one object calling another object via a procedure-calling mechanism. This can also happen over a network via a remote procedure call (RPC). We can separate services so they can operate independently, which is today known as service-oriented architecture (SOA).

Resource-centered architectures:

With the growing service composition to facilitate distributed systems, there was a need to rethink the aspect of web-based distributed systems; new architecture had to be defined to optimise the usage of distributed resources. Resource-centered architectures are robust mechanisms where we can add or remove the resources to be used by applications, based on their usage. Similar resources can be retrieved or modified. These characteristics of resource-centered architectures helped representational state transfer (REST) in web-centric applications gain wide acceptance. Various RESTful operations are showcased in Figure 4.

Event-based architectures: In a large set of distributed systems, we try to decouple the dependencies between the processes. The idea behind this is to view processes running autonomously, and then there is the coordination between these processes which allows communication and data sharing between them. This helps adopt a separation between processing and coordination. In event-based architectures, as shown in Figure 5, after the separation is enabled, processes subscribe to the notification from the other components. These notifications are delivered to only those processes that have subscribed to them. This architecture uses an event bus to match the publisher and subscribers, and communicate the notifications.

System architecture

We have already looked into how the architectural styles are commonly applied in distributed systems. Now let us look at how software components

talk to one another, and how and where they are placed. Deciding on software components, their interaction, and their placement leads to an instance of software architecture, also known as system architecture.

Distributed systems can be categorised into centralised and decentralised architectures. In a centralised architecture, we consider simple server-client architecture, three-tier architecture, and n-tier architecture. When we talk about decentralised architecture, we consider peer-to-peer architecture.

Simple server-client architecture:

Clients make a request to the server; so this request is to be served. The response to that request is served and any permanent changes are stored by the server.

Three-tier architecture:

Here the tier denotes user interface, business logic, and data access.

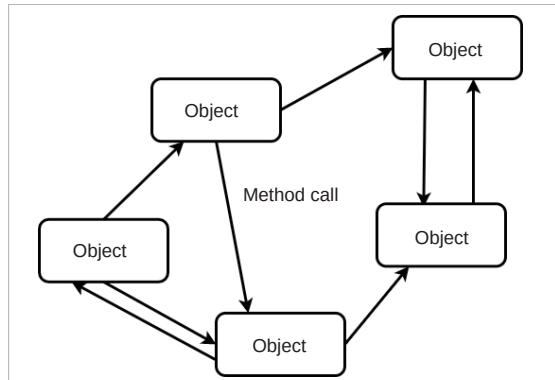


Figure 3: Object-based architecture with various components communicating with one another

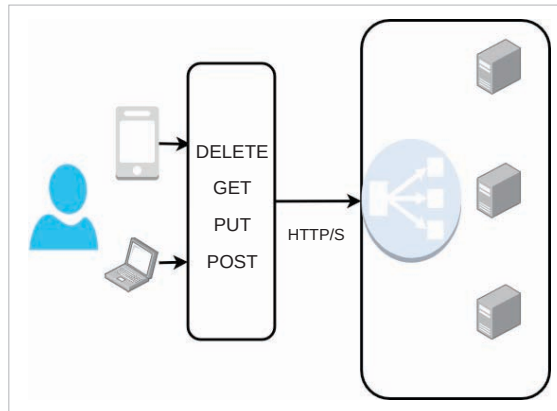


Figure 4: Resource-centred design with various RESTful operations

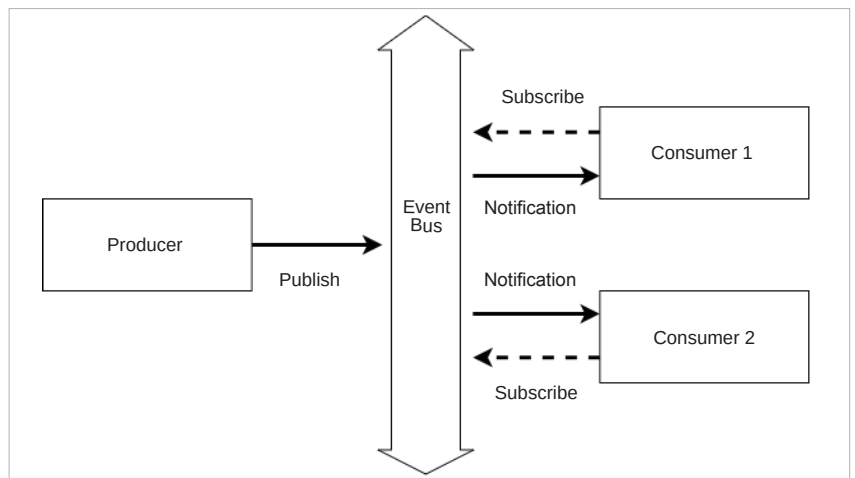


Figure 5: Multiple components in event-based architectural style